

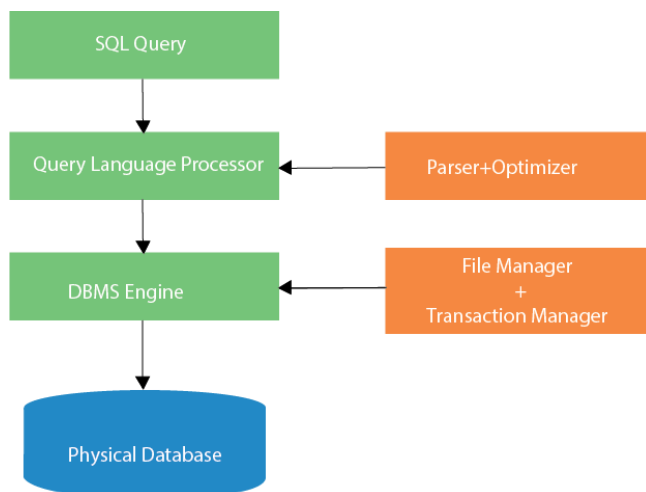
Introduction to SQL

What is SQL?

- SQL stands for Structured Query Language. It is used for storing and managing data in relational database management system (RDMS). In RDBMS data is stored in the form of the tables.
- It is a standard language for Relational Database System. It enables a user to create, read, update and delete relational databases and tables.
- All the RDBMS like MySQL, Informix, Oracle, MS Access and SQL Server use SQL as their standard database language.
- SQL allows users to query the database in several ways, using English-like statements.

How does SQL process work?

When an SQL command is executing for any RDBMS, then the system figures out the best way to carry out the request and the SQL engine determines how to interpret the task. In this process, various components are included. These components can be optimization Engine, Query engine, Query dispatcher, classic, etc.



SQL Query Execution Order



Types of SQL Commands:

SQL Command			
DDL	DML	DCL	TCL
➤ Create ➤ Drop ➤ Alter ➤ Truncate ➤ Rename	➤ Insert ➤ Update ➤ Delete ➤ Select	➤ Grant ➤ Revoke	➤ Commit ➤ Rollback ➤ Save point

Data Definition Language (DDL) Statements: -

It is used to create and modify the structure of database objects in database. Includes statements like:

DDL Syntax

1. Creating A New Database:

```
CREATE DATABASE <Database_Name>
```

2. Dropping Database: Used to drop an existing database.

```
DROP DATABASE <database_name >
```

3. Creating Tables: Used to create a table with a specific structure inside a selected database

```
CREATE TABLE <tablename>
(
  <column name> <data type> [<column constraint>...]
  PRIMARY KEY <column list>
  FOREIGN KEY <column list> REFERENCES <table specifications>
)
```

Ex:

```
CREATE TABLE STAFF  
(  
  STAFF_ID int Primary Key,  
  STAFF_NAME char(20) Not Null,  
  STAFF_SALARY real Null,  
  STAFF_PHOTO image Null  
)
```

4. Altering Table: Used to change the table structure i.e. add column, change data type of a column, or remove existent column.

- ALTER TABLE<table name>
 ADD <column name> <data type>
- ALTER TABLE<table name>
 DROP COLUMN <column name>
- ALTER TABLE<table name>
 ALTER COLUMN <column name> <new data type>

Ex: Add the column STAFF_EMAIL to the table STAFF

```
ALTER TABLE STAFF  
ADD STAFF_EMAIL char(30)
```

5. Dropping Table: Used to drop a table structure and all the data stored in it:

```
DROP TABLE <table name>
```

Ex:

```
DROP TABLE STAFF
```

Common Data Types for table attributes are:

1- Numeric: integer number (INT, SMALLINT AND BIGINT), and floating number (FLOAT, REAL).

2- Character: data types are either fixed length (CHAR (n), where n is the number of character) or variable length (VARCHAR(n)).

3- Boolean: TRUE or FALSE.

4- Timestamp

SQL Constraint: -

Rules enforced on data columns on tables. These are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the database.

The following are commonly used constraints available in SQL:

- **NOT NULL:** A Constraint that ensures that a column cannot have NULL value.
- **DEFAULT:** A Constraint that provides a default value for a column when none is specified.
- **UNIQUE:** A Constraint that ensures that all values in a column are different or NULL.
- **PRIMARY Key:** A Constraint that uniquely identify each row/record in a database table (NOT NULL + UNIQUE)
- **FOREIGN KEY (FK):** A Constraint that ensures referential integrity. A foreign key from 1 table to another is used to link a tuple in the 1st table to a unique tuple in the 2nd table.
- **CHECK:** A constraint that ensures that all values in a column satisfy a certain condition.

Constraints can be specified within the CREATE TABLE statement after the attributes are declared, or they can be added later using the ALTER TABLE command.

Ex1: Consider the following schema of database “University”:

Student (SSN, Name, City, Age, Major)

Course(CrsCode, Name)

Registered(SSN, CrsCode, Semester, Year)

Department(DeptCode, Name)

- Write SQL DDL statement for declaring the University Database and relations. Specify appropriate keys and referential integrity constraints
- Note that: you must ensure that the Age of student cannot be below 18 years.

CREATE database University

--DEPARTMENT Table

CREATE TABLE Department(
DeptCode VARCHAR(5) NOT NULL PRIMARY KEY ,

Name VARCHAR(15) NOT NULL,
)

--Course Table

CREATE TABLE Course

(
CrsCode SMALLINT NOT NULL,
Name VARCHAR(45) UNIQUE,
PRIMARY KEY (CrsCode)
)

--Student Table

CREATE TABLE Student

(
SSN INT NOT NULL,
Name VARCHAR(45),
Age INT CHECK (AGE >= 18),
City VARCHAR(15) DEFAULT 'CAIRO',
Major VARCHAR(5)
)

--Add PK to Student table

ALTER TABLE STUDENT
ADD PRIMARY KEY (SSN),

--OR

ALTER TABLE STUDENT
ADD CONSTRAINT PK1 PRIMARY KEY (SSN) -- constraint name is unique across database

-- Add FK Constraint to Student table

ALTER TABLE STUDENT
ADD CONSTRAINT FK_1 FOREIGN KEY (Major) REFERENCES Department (DeptCode)

--Registered Table

CREATE TABLE Registered

(
SSN INT NOT NULL,
CrsCode SMALLINT NOT NULL REFERENCES COURSE(Crscod),
Semester VARCHAR (45) NOT NULL,
[Year] VARCHAR (45)
)

-- Add pk and fk constraints to Registered Table

ALTER TABLE Registered
ADD PRIMARY KEY (SSN, CrsCode)
ALTER TABLE Registered
ADD FOREIGN KEY (SSN) REFERENCES Student (SSN)
ALTER TABLE Registered
ADD FOREIGN KEY (CrsCode) REFERENCES Course (CrsCode)

ALTER EXAMPLES:

- Add Anew field in “Department” Table to store Number of Student for each Department.
⇒ `ALTER TABLE Department
ADD numberofstudents int`
- Delete “Major” Column from “Student” Table.
⇒ `ALTER TABLE Student
DROP COLUMN Major`
- Change the data type of Column “Name” in table “Student” to be VarChar(100).
⇒ `ALTER TABLE Student
ALTER COLUMN [Name] VARCHAR (100)`
- Add a check constraint to make sure that "DeptCode" will only take one of these codes (IS, CS, IT, DS).
⇒ `ALTER TABLE DEPARTMENT
ADD CONSTRAINT CK_DEPTCODE CHECK (DEPTCODE IN ('IS','CS','IT','DS'))`
- Delete the constraint that you created in the previous example
⇒ `ALTER TABLE DEPARTMENT
DROP CONSTRAINT CK_DEPTCODE`

Data Manipulation Language (DML) Statements:

It is used to manipulate the data (select, insert, delete or update)

1. Inserting New Rows

It is possible to write insert into statements in two ways:

- 1- specifies both the column names and the values to be inserted:

```
INSERT INTO <tablename> [<column list>]  
VALUES (<expression>{[,<expression>]})
```

- 2- if you are adding all values to all columns, you don't need to specify column names

```
INSERT INTO <tablename>  
VALUES (value1,value2,.....)
```

Hint: you should make sure the order of the values is in the same order as the columns in the table.

Example 1:

- ❖ Insert new student into the student table with the following properties:
Student id=1 , Name= Ali, Age=19, and City= 'Cairo'

```
INSERT INTO Student (SSN,Name,Age,City) VALUES (1,'Ali',19,'Cairo')
```

```
INSERT INTO Student VALUES (2,'Wafaa',19,'Cairo',null,null)
```

```
////////////////////////////////////
```

```
insert into Student values (3,'Wafaa',19,'Cairo','IS',null) → this raise an error as major is the FK  
and not inserted yet into department ... you should insert this row first into Department  
table
```

```
INSERT INTO department values ('IS','Information System')
```

2. Updating Values in Rows

```
UPDATE <tablename>
```

```
SET <column name>=<expression>, <column name>=<expression>
```

```
[WHERE <Condition>]
```

Example 1:

- ❖ Increase the age of students by 1

```
update Student  
set age=age+1
```

Example 2: (Using Where Clause)

- ❖ Change the name of the Student with SSN 1 to be 'AHMED'

```
update Student  
set name='ahmed'  
where SSN=1
```

Example 3: (Update more than one column with a condition)

- ❖ Change the name of the STAFF with id 1 back to 'ALI' and let his age be 18

```
update Student  
set name='Ali',  
age=18  
where SSN=1
```

3. Deleting rows from a table

```
DELETE  
FROM <table name>  
[WHERE <Condition>]
```

Example 1:

❖ Delete all rows of table Student (Don't try it)

```
delete from Student
```

Example 2:

❖ Delete the Student with SSN 2

```
delete from Student  
where SSN=2
```

Example 3: (Using more than one condition)

❖ Delete the Student that has an id 1 or 2

```
delete from Student  
where SSN=2 or  
SSN=1
```

BETTER STATEMENT THAT DOES THE SAME JOB OF THE PREVIOUS ONE :

```
delete from Student  
where SSN IN (1,2);
```

4. Retrieve (Select) data

4.1. Select data from a single table

```
Select <columns>  
FROM <table name>  
[WHERE <Condition>]
```

Simple Select Statements SELECT <Column list> FROM <table names> [WHERE <Condition>]	Example 1: (using northwind database): ❖ Retrieve all information of all products SELECT * FROM [dbo].[products]
Example 2: ❖ Retrieve Product id and Product name of all products SELECT ProductID, ProductName FROM products	Example 3: ❖ Get all PRODUCTS where their names contain the letter H SELECT * FROM products where ProductName like '%h%'
Example 4: (Using DISTINCT Keyword) ❖ Get all unique CATOGRIES of the products SELECT DISTINCT CategoryID FROM [dbo].[products] ❖ Get all PRODCUTS where their Category is either 1 or 2 SELECT * FROM [dbo].[products] WHERE categoryid=1 or categoryid= 2	

4.2. Select data from multiple tables

A JOIN clause is used to combine rows from two or more tables, based on a related column between them.

Different Types of SQL JOINS

- **(INNER) JOIN:** Returns records that have matching values in both tables
- **LEFT (OUTER) JOIN:** Returns all records from the left table, and the matched records from the right table
- **RIGHT (OUTER) JOIN:** Returns all records from the right table, and the matched records from the left table
- **FULL (OUTER) JOIN:** Returns all records when there is a match in either left or right table



Join Examples (Using Northwind DB):

Example 1:

Get the names, prices and names of companies that supply these products.

```
SELECT Products.ProductName,Products.UnitPrice,Suppliers.CompanyName
FROM Products inner join Suppliers
On Products.SupplierID = Suppliers.SupplierID
--OR
SELECT Products.ProductName,Products.UnitPrice,Suppliers.CompanyName
FROM Products, Suppliers
WHERE Products.SupplierID = Suppliers.SupplierID
```

Example 2:

Get the names of products categories and the name of the company that supply these categories.

```
Select Categories.CategoryName,Suppliers.CompanyName
from Categories inner join Products
on Categories.CategoryID =Products.CategoryID
inner join Suppliers
on Products.SupplierID=Suppliers.SupplierID;
```

--OR

```
SELECT Categories.CategoryName, Suppliers.CompanyName
FROM Categories, Products, Suppliers
WHERE Suppliers.SupplierID = Products.SupplierID
      AND Products.CategoryID = Categories.CategoryID
```

Example 3:

Get names of all the customers and if they place any orders show their order date.

```
SELECT Customers.CompanyName, Orders.OrderDate
FROM Orders RIGHT OUTER JOIN Customers
ON Orders.CustomerID = Customers.CustomerID
```

OR

```
SELECT Customers.CompanyName, Orders.OrderDate
FROM Customers LEFT OUTER JOIN Orders
ON Customers.CustomerID=Orders.CustomerID
```

Example 4:

Get names of all products and all categories ,showing which products belong to which categories if any:

```
SELECT Products.ProductName, Categories.CategoryName
FROM Products FULL OUTER JOIN Categories
ON Products.CategoryID =Categories.CategoryID
```

Note: you need to insert a new Category and a new product to be able to know the changes at the results

- `INSERT INTO Categories (CategoryName) Values('New Category')`
 - `INSERT INTO Categories (CategoryName) Values('New Category')`
-

4.3. Select data (set operations, Aggregations, group by and having)

Set operations: allow the results of multiple queries to be combined into a single result set. Set operators include UNION, INTERSECT, and EXCEPT.

1- UNION OPERATOR: combines the results of two SQL queries into a single table of all matching rows. The two queries must result in the same number of columns and compatible data types to unite. Any duplicate records are automatically removed unless UNION ALL is used.

2- INTERSECT OPERATOR: takes the results of two queries and returns only rows that appear in both result sets. The INTERSECT operator removes duplicate rows from the final result set. The INTERSECT ALL operator does not remove duplicate rows from the final result set.

3- EXCEPT OPERATOR: takes the distinct rows of one query and returns the rows that do not appear in a second result set. The EXCEPT ALL operator does not remove duplicates.

Aggregate functions are used to summarize information from multiple tuples into a single-tuple summary.

Grouping is used to create subgroups of tuples before summarization

Example1: Get the customers IDS who didn't make any order.	<pre>SELECT CustomerID FROM Customers EXCEPT SELECT CustomerID FROM Orders</pre>
Example 2: Get all names of suppliers who supply products of category named 'Beverages' and products of category named 'Condiments'	<pre>SELECT Suppliers.CompanyName FROM Products, Suppliers, Categories WHERE Suppliers.SupplierID = Products.SupplierID AND Products.CategoryID =Categories.CategoryID AND Categories.CategoryName ='Beverages' INTERSECT SELECT Suppliers.CompanyName FROM Products, Suppliers, Categories WHERE Suppliers.SupplierID =Products.SupplierID AND Products.CategoryID =Categories.CategoryID AND Categories.CategoryName ='Condiments'</pre>

Example 3: For a mass-mailing purpose, get the list of all names, addresses, cities and postal codes known to the database and remove duplicates from the result if any (these data are stored in the customers and employees tables).	<pre>SELECT ContactName AS Name,Address, City,PostalCode FROM Customers UNION SELECT FirstName+' '+LastName AS Name, Address, City,PostalCode FROM Employees</pre>
Example 4: For each product ordered in March 1998, get the product id, number of orders for that product and the total quantity ordered. Give suitable name(s) for any unnamed displayed column(s). Sort by total quantity descendingly	<pre>SELECT ProductID, count(od.orderid) as numOfOrders, Sum(quantity) as totalQuantityOrdered FROM [Order Details] od, orders o where od.OrderID = o.OrderID and year(OrderDate) = '1998' and month(OrderDate) = '3' GROUP BY ProductID Order by TotalQuantityOrdered desc</pre> Comments: 1. All the columns in the select with the aggregate function should be included in the group by clause. In the example, the GROUP BY will give you one value (the numer of orders) for each ProductID. 2. In this query, if we have a column next to the aggregate function in the select caluse, we have to write a GROUP BY this column. Otherwise, an error will be given because we are trying to tell the DBMS to show ProductID "Multiple Values" next to an aggregate function. 3. If we removed the other column(s) in the select and the GROUP BY and keep only the aggregate functions. The SUM/COUNT will get us the sum of quantities and total number of orders for the whole table at once.
Example 5: Get the supplier names (Contact names) and the total number of supplied products for each supplier, for only those suppliers that supplied more than 3 products. Give suitable name(s) for any unnamed displayed column(s).	<pre>SELECT ContactName, Count (ProductID) AS NumberOfSuppliedProducts FROM Products, Suppliers where Products.SupplierID = Suppliers.SupplierID GROUP BY ContactName HAVING Count (ProductID) > 3</pre> Comments: 1. HAVING, this clause could be written only in case you have a GROUP BY clause. It is a filtering condition that will be applied on the resultant groups, to filter the groups which do not satisfy the given condition. 2. Conditions which are applied to the data before grouping (the raw data not the aggregated groups) , should be put in the where clause not the HAVING.

Example 6: For all customers with at least 15 orders after 1996, get the customer id, total number of orders, the last order date and the average unit price for products in his/her orders . Give suitable name(s) for any unnamed displayed column(s). Display the result starting from customers with the highest number of orders.	<pre> SELECT CustomerID, COUNT (o.OrderID) AS NumberOfOrders, MAX (OrderDate) AS LastOrderDate, Avg(Unitprice) as AveragePrice FROM Orders o, [order details] od WHERE o.OrderID = od.orderid and YEAR (OrderDate) > 1996 GROUP BY CustomerID HAVING COUNT (o.OrderID) > 15 and Avg(unitPrice)> 25 ORDER BY COUNT (o.OrderID) DESC </pre>
---	--

4.4. Select data (Nested Queries)

A **Subquery** is a query within another query. Subqueries can reside in the WHERE Clause, FROM Clause or SELECT Clause.

1. Using Sub-queries in the where clause – IN

1. Display the first name, title of all employees who live in UK and have the same job (title) as Janet or Steven. Sort the result by the first name of the employees in a descending order.

```

SELECT FirstName, Title
FROM Employees
WHERE Country = 'UK'
AND Title IN (SELECT Title
              FROM Employees
              WHERE FirstName IN ('Janet', 'Steven'))
ORDER BY FirstName DESC

```

2. Get a list of orders (with order dates) that were processed by employees who are not sales representatives:

```

SELECT [OrderID], [OrderDate]
FROM [Orders]
WHERE [EmployeeID] IN
(SELECT [EmployeeID] FROM [Employees]
 WHERE [Title]<>'Sales Representative')

```

2. Using Sub-queries in the where clause –Exists

1. Return a list of customers who has orders shipped to UK

```
select CustomerID, CompanyName
from customers as a
where exists
(
    select * from orders as b
    where a.CustomerID = b.CustomerID
    and ShipCountry = 'UK'
)
```

OR using join

```
select CustomerID, CompanyName
from customers, orders
where CustomerID = CustomerID
and ShipCountry = 'UK'
```

3. Using Sub-queries in the where clause –Not Exists

1. Get the distinct names for products that are always ordered with quantity greater than 10.

```
SELECT DISTINCT ProductName
FROM [Products]
WHERE NOT EXISTS
(SELECT * FROM [Order Details]
WHERE [Order Details].[ProductID]=[Products].[ProductID]
and QUANTITY<10);
```

2. Get the customer name and title for customers that have not placed any orders.

```
SELECT ContactName,ContactTitle
FROM Customers AS c
WHERE Not EXISTS(SELECT *
FROM [Orders] AS o
WHERE o.CustomerID = c.CustomerID)
```

Note:

- When using subquery in the where clause, take care when using operators; If the subquery returns more than one value you can't use operator such as =, but you may use the IN operator.
- (IN) predicate can either return TRUE, FALSE or NULL:
 - TRUE is returned when the required value is NOT NULL and can be found.

- FALSE is returned when the required value is NOT NULL and is NOT found and the list DOES NOT contain NULL values
- NULL is returned when the required value is NULL, or when the required values is NOT NULL and was not found in the list and the list contains at least one NULL value
- EXISTS condition is considered "to be met" if the sub query returns at least one row, it always returns TRUE or FALSE
 - TRUE as soon as it finds only a single matching row in the sub query
 - FALSE, if it finds none.
- NOT EXISTS will return TRUE only if no row is returned from the sub query.

4. Using Sub-queries in the where clause – “All”

1. Get the last name, all employees who were hired before all employees working as ‘Sales Managers’ and ‘Inside Sales Coordinators’.

```
SELECT LastName
FROM Employees
where HireDate < ALL (SELECT HireDate
                      FROM Employees
                      WHERE Title IN ('Sales Manager', 'Inside Sales Coordinator'))
```

5. Using Subquery in FROM Clause “Represents a table”

1. Get the supplier name (contact name) and the supplied product name for products having unit price less than 7. Fully optimize your query to minimize the cost of accessing any unneeded columns and/or rows from the relevant tables.

```
SELECT S.ContactName, P.ProductName
FROM (SELECT ContactName, SupplierID FROM Suppliers) AS S ,
     (SELECT ProductName, SupplierID, UnitPrice FROM Products WHERE
      UnitPrice < 7) AS P
Where S.SupplierID = P.SupplierID
```

Note:

- Mainly we use this way to shrink the size of the table and decrease the cost of working with the whole table (considering all columns in it). So, we choose only the set of columns we need from the table(s) related to our query.
- As shown in the above example, we can give an alias to the subquery and use it in the main query.